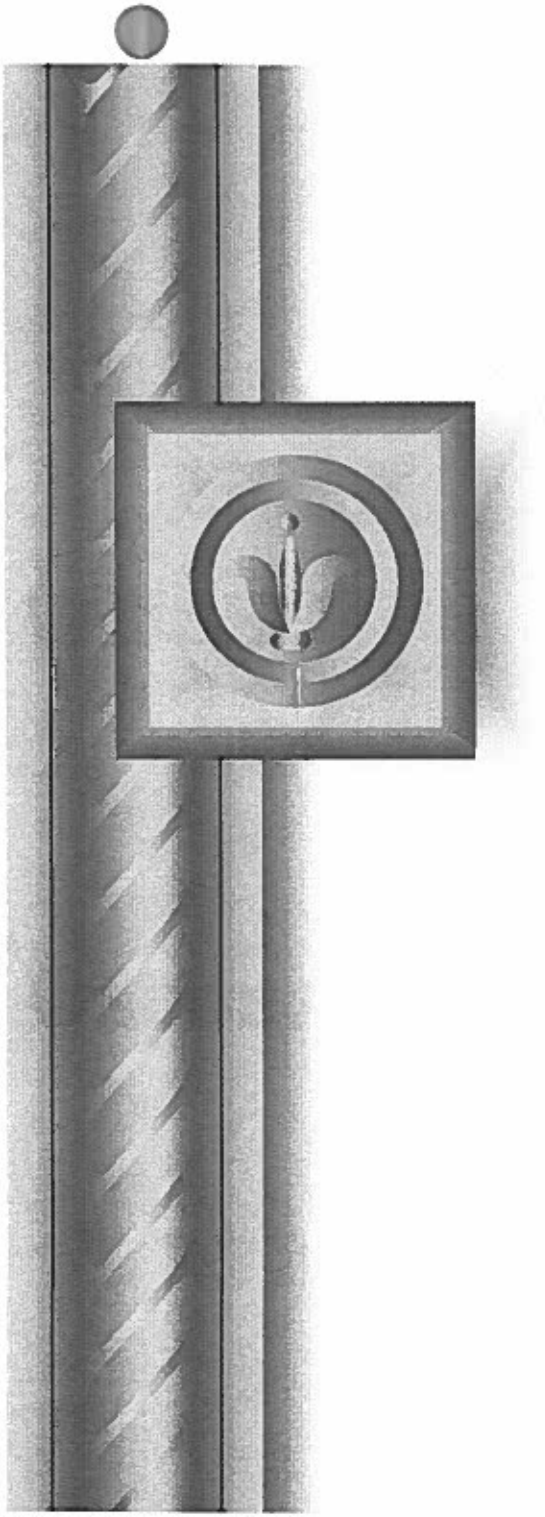


Client/Server Tic-Tac-Toe

Oscar A. Smalls, Jr.

11/14/2006

Functionality

- **Test for win, lose, and draw**
- **Display "Play Again?" button at end of game**
- **Display "Quit?" button during game**
- **Winner can choose 'X' or 'O' at end of game**

Progress

- **Currently working on conversion**
  - **Conversion to be done by Nov. 21**
- **Testing**
  - **Clients on different computers**
  - **Eliminating bugs**
  - **Testing to be done by Nov. 28**

Oscar A. Smalls, Jr.
COIN 435: Advanced Networking - Final Project Progress Report
November 14, 2006

INTRODUCTION

The purpose of this final project is to convert an existing multithreaded, client/server Tic-Tac-Toe program that was written in Java to a multithreaded client/server version written in C#.

FUNCTIONALITY

1. The server will test for a win, loss, or draw on each move in the game and send a message to each client that indicates the result of the game when the game is over.

2. The client will display a button that when clicked allows the client to play another game. The button should be enabled only when a game completes. The other client should be notified that a new game is about to begin so its board and state can be reset.

3. The client will provide a button that allows a user to terminate the program at any time. When the user clicks the button, the server and the other client should be notified.

4. The winner of a game can choose game piece X or O for the next game. X always goes first.

PROGRESS

I am currently working on the conversion from Java to C#. The conversion should be completed by November 21st. After the conversion, the client/server sections will need to be tested on separate computers to ensure that it will work in a networked environment. Any bugs carried over from the original Java program will be worked out by November 28th.

Client/Server Tic-Tac-Toe

Oscar A. Smalls, Jr.

11/20/2007

Functionality

- Test for win, lose, and draw
- Display "Play Again?" button at end of game
- Display "Quit?" button during game
- Winner can choose 'X' or 'O' at end of game

Tic-Tac-Toe Server

- Console based (no GUI)
- Two classes
  - Server class
    - Starts Listener and Player threads
    - Validate moves; Notifies clients
  - Player class
    - Gets move from client
    - Asks server to validate move
    - Sends messages to client

Tic-Tac-Toe Server

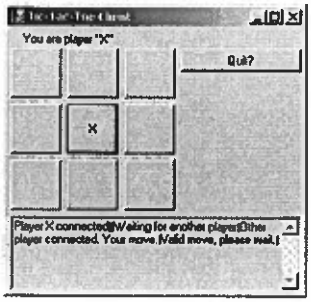
- Listener thread
  - Waits for Player X to connect
  - Accepts Player X socket
  - Starts Player X thread with socket
  - Suspends Player X thread until Player O connects
  - Waits for Player O to connect
  - Accepts Player O socket
  - Starts Player O thread with socket

Tic-Tac-Toe Server

Tic-Tac-Toe Client

- Buttons used to accept user input
- Final version asks for server ip
- Two classes
  - Form class
    - Process messages from server
    - Displays messages to user
  - Square class
    - Put mark in square
    - Gets mark in square

Tic-Tac-Toe Client



11 20 2007 Oscar A. Smalls, Jr. 7

Progress

- **Currently finishing up conversion**
- **Next Step: Testing**
  - **Clients on different computers**
  - **Eliminating bugs**
  - **Testing to be done by Nov. 28**

11 20 2007 Oscar A. Smalls, Jr. 8

Oscar A. Smalls, Jr.
COIN 435: Advanced Networking - Final Project Report
December 11, 2006

INTRODUCTION

The purpose of this final project is to convert an existing multithreaded, client/server Tic-Tac-Toe program that was written in Java to a multithreaded client/server version written in C#.

FUNCTIONALITY

1. The server will test for a win, loss, or draw on each move in the game and send a message to each client that indicates the result of the game when the game is over.

2. The client will display a button that when clicked allows the client to play another game. The button should be enabled only when a game completes. The other client should be notified that a new game is about to begin so its board and state can be reset.

3. The client will provide a button that allows a user to terminate the program at any time. When the user clicks the button, the server and the other client should be notified.

4. The winner of a game can choose game piece X or O for the next game. X always goes first.

PROGRESS

This project has been completed. After converting the Java client/server programs to C#, the C# client/server programs were tested thoroughly on the same computer to make sure that the functionality described above worked. The client/server programs were then enhanced to allow the server program to reside on a computer separate from the client programs.

DOCUMENTATION

1. Run the TicTacToeServer.exe application on one computer. The server will start as a console application and display its IP address.

2. Run the TicTacToeClient.exe application on another computer. Enter the IP address of the server and press the "Connect" button. The client will create a socket and connect to the listener running on the server. When the server accepts the socket connection, the server will create and start a thread to work with this client application. Since this is the first client connecting to the server, this client will be player "X" and its thread will be suspended until player "O" connects.

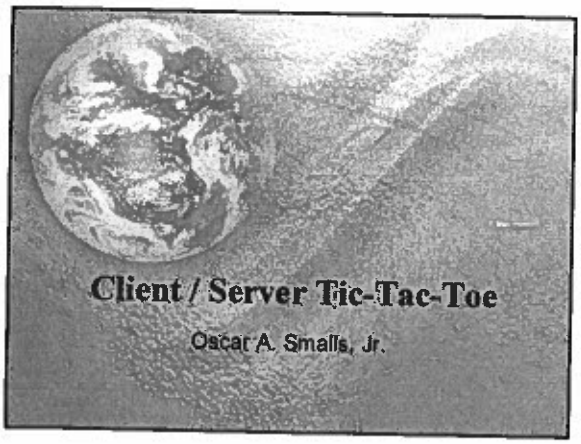
3. Run the TicTacToeClient.exe application on another computer. Enter the IP address of the server and press the "Connect" button. The client will create a socket and connect to the listener running on the server. When the server accepts the socket connection, the server will create and start a thread to work with this client application. Since this is the second client connecting to the server, this client will be player "O". The server will notify the thread for Player "X" to continue.

4. When it is your player's turn, press the buttons on the game board to make a move. Making a move sends to the server an integer of the value of the square selected. Square values are from 0 through 8. The server will check each move, and if it is valid, it will notify the other player thread of the move so that its game board can be updated. If the move is invalid, the server will notify you that the move is invalid. The server notifies clients by sending string messages like "opponent moved" if the client should perform a specific action or "please wait" if the message does not need to be processed but just displayed to the user.

5. The server will continue to give each client a turn until it detects that one of the clients has won or there are no more moves to make. When it detects an end-of-game condition, it notifies both clients through string messages.

6. If the user selects a button to end the game or any other button that is not a square on the game board, the client application sends an integer value to its player thread running on the server. A value of -1 means that player "X" wants to quit the game. A value of -2 means player "O" wants to quit the game. A value of -3 means that the winner of the last game wants to play again. If the winner chooses to play again, the server sends a message to the client to force the client to enable the "Choose to be X" and "Choose to be O" buttons and disable any unnecessary buttons. Pressing the "Choose to be X" button sends a -1 to the server while pressing the "Choose to be O" button sends a -2 to the server. The server then processes the choice made.

7. After both players leave the game, the server will listen for two more players to connect and then play the game with those players.



Functionality

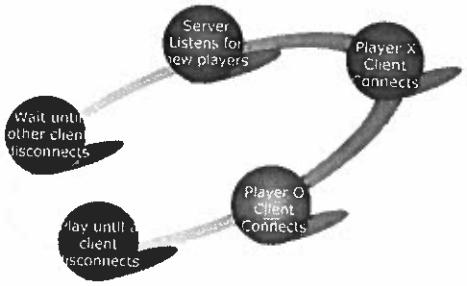
- Test for Win, Lose, or Draw
- Display "Play Again?" button
- Display "Quit?" button
- Winner chooses "X" or "O"

Progress

Phase 1 | Phase 2 | Phase 3

Conversion → Debugging → Testing

Process



Class Diagram

Square Class

```
// utility method to manipulate set mark in square's
// button in the event-dispatch thread
private delegate void setMarkDelegate(string mark);

// put mark in Square's button
internal void setMark(string mark)
{
   if (button.InvokeRequired)
   {
      button.Invoke(new setMarkDelegate(setMark), new object[] { mark });
   }
   else
   {
      button.Text = mark;
   }
}
```

Form Class

```
else if (message == "Opponent moved")
        {
            // get move location and update board
            try
            {
                int location = input.ReadInt32();
                int row = location / 3;
                int column = location % 3;

                setMark(board[row, column], (myMark == X_MARK ? O_MARK : X_MARK));

                displayMessage("Opponent moved. Your turn.");
                myTurn = true;
            }
            catch (IOException ioException)
            {
                Console.WriteLine(ioException.StackTrace);
            }
        }
```

Player Class

```
// if player X, wait for another player to arrive
        if (mark == X_MARK)
        {
            write("disableButtons");
            write("Waiting for another player");

            // wait for player O
            lock (this)
            {
                while (suspended)
                    Monitor.Wait(this);
            }

            // send message that other player connected and
            // player X can make a move
            write("enableButtons");
            write("Other player connected. Your move.");
        }
```

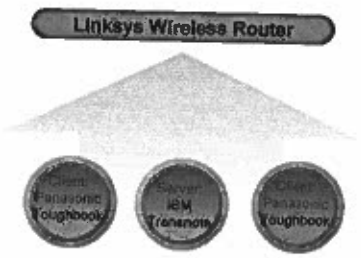
Server Class

```
while (true)
{
   // wait for connection, create Player X, start Player X thread
   players[PLAYER_X] = new Player(this, listenerSocket.Accept(), PLAYER_X);
   playerThread[PLAYER_X] = new Thread(new ThreadStart(players[PLAYER_X].run));
   playerThread[PLAYER_X].Start();

   // wait for connection, create Player O, start Player O thread
   players[PLAYER_O] = new Player(this, listenerSocket.Accept(), PLAYER_O);
   playerThread[PLAYER_O] = new Thread(new ThreadStart(players[PLAYER_O].run));
   playerThread[PLAYER_O].Start();

   // Player X is suspended until Player O connects
   // Resume player X now
   lock (players[PLAYER_X])
   {
      players[PLAYER_X].setSuspended(false);
      Monitor.Pulse(players[PLAYER_X]);
   }
}
```

Network Diagram



Remote Procedure Calls (RPC)

Oscar A. Smalls, Jr.

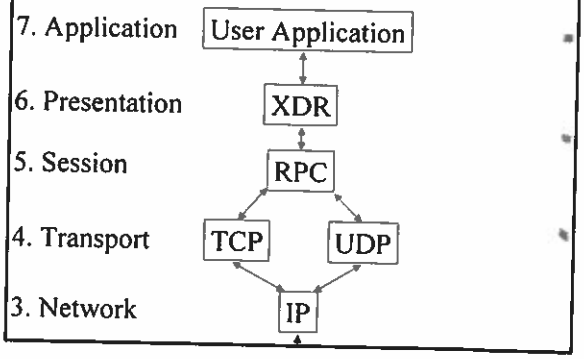
Introduction

- Sun Microsystems' Open Network Computing (ONC) group provides the only freely available RPC
- ONC RPC is used with:
  - NFS
  - NIS (Sun's Network Information System)
  - User Programs

Topics of Discussion

- RPC and the OSI Reference Model
- RPC Application Development
- RPC and Portmapper

RPC and the OSI model



RPC Application Development

1. Specify the protocol for communication
2. Compile the protocol to generate header file, client/server stubs, and XDR routines
3. Develop server and client applications
4. Compile, Link, and Execute

Specify Protocol (math.x)

```
struct input_struct {
  long operand1;
  long operand2;
};

typedef long output_type;

program ADD_PROG {
  version ADD_VERS {
    output_type ADD_PROC(input_struct) = 1; /* Procedure Number */
  } = 1; /* Version Number */
} = 11111; /* Program Number */
```

Compile Protocol (math.x)

- rpcgen math.x
  - Generate header file for stubs and applications
    - math.h
  - Generates stubs
    - math_clnt.c (stub called by client)
    - math_svc.c (server stub that calls server procedure)
  - Generate XDR filter if necessary
    - math_xdr.c (for input_struct and output_type)

Header File (math.h)

```
#define ADD_PROG 11111
#define ADD_VERS 1

#define ADD_PROC 1
extern output_type * add_proc_1(input_struct *,
  CLIENT *); /* stub called by client */
extern output_type * add_proc_1_svc(input_struct
  *, struct svc_req *); /* server procedure */
```

Server Procedure (server.c)

```
#include "math.h"

output_type *add_proc_1_svc(input_struct *in, struct svc_req *svc) {
 static output_type out;
 out = in->operand1 + in->operand2;
 return(&out);
}

output_type *mult_proc_1_svc(input_struct *in, struct svc_req *svc) {
 static output_type out;
 out = in->operand1 * in->operand2;
 return(&out);
}
```

Client Program (client.c)

```
#include "math.h"
int main(int argc, char **argv) {
    CLIENT *add_client;
    input_struct in;
    output_type *out;
    add_client = clnt_create(argv[1], ADD_PROG, ADD_VERS, "tcp" );
    in.operand1 = atol(argv[2]);
    in.operand2 = atol(argv[3]);
    out = add_proc_1(&in,add_client), /* call client stub */
    printf("Added = %ld\n", * out);
    clnt_destroy(add_client);
    return 0;
}
```

Compile & Link

```
# gcc -c server.c
# gcc -c client.c
# gcc -c math_svc.c
# gcc -c math_clnt.c
# gcc -c math_xdr.c
# gcc -o server server.o math_svc.o math_xdr.o
# gcc -o client client.o math_clnt.o math_xdr.o
```

Execute Server

```
# ./server &
# rpcinfo -p localhost
   program   vers   proto   port
    100000    2      tcp     111   portmapper
    100000    2      udp     111   portmapper
    100024    1      udp   32769   status
    100024    1      tcp   58226   status
    11111     1      udp     926
    22222     1      udp     926
    11111     1      tcp     929
    22222     1      tcp     929
```

Execute Client

```
# ./client localhost 2 3
Added = 5
```

RPC and Portmapper

- Portmapper on every host is well known. It is always assigned to port 111.
- The client connects to Portmapper to find out which port it should use to access server process.
- The client and server now open a communication path to perform remote procedure execution.

Review

- Reviewed OSI model of RPC System
- Developed an RPC Application
- Reviewed the use Portmapper with RPC

For more information on RPC

- Power Programming with RPC by John Bloomer (1992)
- "Remote Procedure Calls for Distributed Systems" by Ed Petron (April 15, 2001) http://www.informit.com/articles/article.aspx?p=21136&seqNum=4&rl=1
- "Remote Procedure Call" by Christian Poellabauer (Spring 2007) http://www.cse.nd.edu/~cpoellab/teaching/cse30264/lecture14.pdf